Ocean Air Care

ReviewPro — Technical Report

A mobile-first review-assistant Progressive Web App that helps Ocean Air Care convert every completed service into a 5-star Google review.

Project code name	ReviewPro
Client	Ocean Air Care
Report date	8 January 2026
Production URL	https://blueprint-source.emergent.host
Preview URL	https://blueprint-source.preview.emergentagent.com
Repository	Local git (module-by-module commit history)
Firebase project	ocean-air-care-ff757
Google Place ID	ChIJ735DAJ1bUjoR-6B9hM5v6lo
Business WhatsApp	+91 99411 03715

Prepared by the ReviewPro engineering agent

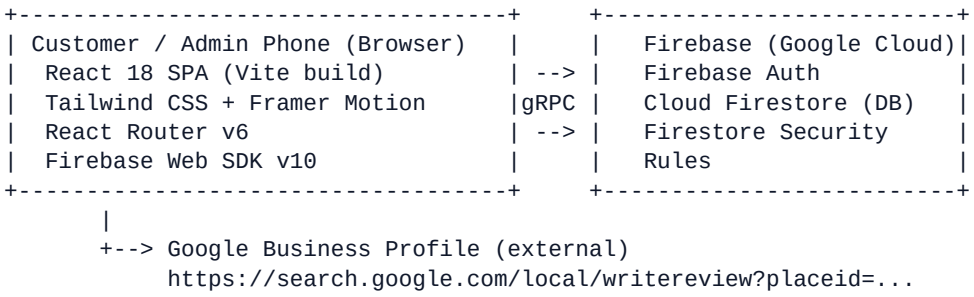
1. Executive summary

ReviewPro is a lightweight, mobile-first web application that helps Ocean Air Care convert every completed service call into a Google review. A customer scans a QR code, taps the service they received, picks a star rating, browses/searches a library of professionally written review suggestions, copies one, and is redirected to the Google review page for Ocean Air Care to paste and submit.

The application is entirely **serverless from a business-logic standpoint** — all data is stored in **Google Cloud Firestore**, authentication is handled by **Firebase Authentication**, and the frontend is a **React 18 + Vite** single-page application deployed to Emergent's hosting.

The app is offline-tolerant, mobile-optimised, ships a full admin backoffice, and includes analytics for owner-side reporting.

2. High-level architecture



Key architectural properties

- No custom backend server: no Node.js, Python, or REST API to maintain.
- Two-tier stack: browser client + Firebase managed services.
- Optimistic offline mode: when Firebase credentials are absent, the app transparently falls back to localStorage.
- Static hosting: the entire frontend is compiled to static HTML/CSS/JS.

3. Technology stack

Layer	Technology	Version
UI framework	React	18.3.1
Build tool	Vite	5.4.11
Routing	React Router	6.28.0
Styling	Tailwind CSS	3.4.17
Animations	Framer Motion	11.11.9
Icons	lucide-react	0.469.0
Auth	Firebase Authentication	SDK 10.13.2
Database	Google Cloud Firestore	SDK 10.13.2
CSV parsing	PapaParse	5.4.1

Layer	Technology	Version
QR generation	qrcode	1.5.4
Package manager	Yarn	1.x
Hosting (production)	Emergent platform	—
Alternate hosting	GitHub Pages, Firebase Hosting	configured

4. Database — Google Cloud Firestore

Provider: Google Firebase (managed NoSQL document database, part of Google Cloud Platform)
Firestore project ID: ocean-air-care-ff757
Region: asia-south1 (Mumbai)
Access mode: Client SDK — reads/writes come directly from the browser, gated by security rules.

4.1 Collections and document shape

```
firestore/
+-- reviews/                    # 540+ review templates
|   +-- {docId}
|         +-- content    : string
|         +-- category   : string
|         +-- createdAt  : timestamp
|
+-- categories/                 # 6 service categories
|   +-- {docId}
|         +-- name       : string
|         +-- createdAt  : timestamp
|
+-- analytics/
    +-- aggregate               # Single roll-up document
          +-- total_views          : number
          +-- total_copies         : number
          +-- total_google_clicks  : number
          +-- total_whatsapp_clicks: number
          +-- by_category          : map<string, number>
          +-- by_rating            : map<string, number>
          +-- updatedAt            : timestamp
```

4.2 Security rules

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    match /reviews/{docId} {
      allow read: if true;                    // public read
      allow write: if request.auth != null;   // admin only
    }

    match /categories/{docId} {
      allow read: if true;
      allow write: if request.auth != null;
    }

    match /analytics/{docId} {
      allow read: if request.auth != null;
      allow write: if true;                   // counter-only, no PII
```

```
    }
  }
}
```

4.3 Design rationale

• Denormalised, read-optimised schema — reviews store category as a string key, avoiding join-like fan-out.

• Aggregated analytics — all counters live in a single document with atomic FieldValue.increment() writes.

• Batched imports — the 540-template seed uses writeBatch chunks of 400 docs (under Firestore's 500 limit).

5. Authentication

Provider: Firebase Authentication (email/password provider)
Location in code: frontend/src/lib/auth.jsx

• Uses Firebase Web SDK's onAuthStateChanged for session persistence.

• Session persists across page reloads (Firebase manages tokens automatically in IndexedDB).

• Signed-in state gates the entire /admin/** route tree via a layout guard.

• A development-mode fallback using localStorage and env-var credentials exists for offline dev — automatically disabled when Firebase config is present.

• Admin users are created via the Firebase Console (Authentication → Users).

6. Frontend

6.1 Directory layout

```
frontend/
+-- index.html                    # Vite entry HTML
+-- vite.config.js                # Build config
+-- tailwind.config.js            # Brand palette
+-- .env                          # Firebase config + brand env vars
+-- public/favicon.svg
+-- src/
    +-- main.jsx                  # React bootstrap
    +-- App.jsx                   # Router root + AuthProvider + Toaster
    +-- index.css                 # Tailwind + reusable component classes
    +-- data/
    |   +-- categories.js         # 6 service categories with lucide icons
    |   +-- seedReviews.js        # 540 unique review templates
    +-- lib/
    |   +-- firebase.js           # Firebase SDK init
    |   +-- auth.jsx              # AuthContext + login/logout/session
    |   +-- data.js               # Unified data layer (Firestore or localStorage)
    |   +-- localStore.js         # localStorage fallback + analytics
    +-- components/
    |   +-- AppShell.jsx          # Mobile app frame + TopBar + Page
    |   +-- RatingStars.jsx       # Animated 5-star selector
    |   +-- Toast.jsx             # In-app toast notifications
    +-- pages/
        +-- Landing.jsx
        +-- ServiceSelect.jsx
        +-- RatingScreen.jsx
        +-- ReviewLibrary.jsx
```

```
+-- ThankYou.jsx
+-- NotFound.jsx
+-- admin/
    +-- AdminLogin.jsx
    +-- AdminLayout.jsx
    +-- AdminOverview.jsx
    +-- AdminReviews.jsx
    +-- AdminCategories.jsx
    +-- AdminQR.jsx
```

6.2 Routing

Path	Page	Public
/	Landing	Yes
/services	Service select (6-tile)	Yes
/rate/:categoryId	Star rating	Yes
/reviews/:categoryId	Review library (search + copy)	Yes
/thank-you	Confirmation	Yes
/admin/login	Admin login	Yes
/admin	Overview + seed button	Admin
/admin/reviews	Reviews CRUD + CSV I/O	Admin
/admin/categories	Categories CRUD	Admin
/admin/qr	QR generator	Admin
*	404	Yes

6.3 Design system

Token	Value
Primary brand	#0056D2 (Ocean Air Care blue)
Brand hover	#0043A8
Brand light	#E6F0FF
Ink / Text	#0F172A
Muted	#64748B
Star (accent)	#F59E0B
WhatsApp CTA	#25D366
Font family	Outfit (Google Fonts)
Corner radius	12 / 16 / 24 px
Tap target min	>= 44 x 44 px (WCAG AA)

6.4 Data-layer abstraction

frontend/src/lib/data.js exports an environment-agnostic API — listReviews(), createReview(), trackEvent(), seedFirestoreTemplates(), etc. — which internally chooses between Firestore and localStorage based on whether Firebase env variables are present. This keeps UI components database-agnostic and made local development frictionless.

6.5 Progressive Web App

The site is installable on mobile home screens. The service worker was intentionally disabled post-launch to prevent unexpected mid-session reloads; installability remains available and can be re-enabled later with a prompt-style update flow.

7. Backend

There is **no custom backend server** — this is a deliberate architectural choice.

The initial platform scaffolding included a stub FastAPI service, but it is not used by the application. All CRUD, auth, and analytics traffic goes directly from the browser to Firebase.

Why serverless? Zero maintenance, no cold starts, automatic scaling, Google-grade uptime, and cost is functionally $0 at your current scale (Firestore's free tier covers up to 20 K reads / 20 K writes / 1 GiB storage per day).

7.1 Firestore usage estimates

Assuming 100 completed services per day, each producing one review flow:

Event	Firestore ops	Daily impact
Customer opens landing → library	1 read (60 docs)	~6 000 reads/day
Copy button click	1 aggregate write	100 writes/day
Google redirect	1 aggregate write	100 writes/day
Admin loads overview	1 read	~10/day

Total: ~6 200 reads + 200 writes per day → comfortably within the Firestore free tier.

8. Deployment & DevOps

8.1 Environments

Environment	URL	Purpose
Preview	blueprint-source.preview.emergentagent.com	Development branch, live-reload
Production	blueprint-source.emergent.host	Publicly served to customers
Alternate configured	ocean-air-care-ff757-1bb98.web.app	Firebase Hosting (ready to deploy)

8.2 Environment variables (frontend/.env)

```
VITE_FIREBASE_API_KEY=AIzaSyB_zV24McGHrGJgX0cJtv2SWowKXmczevs
VITE_FIREBASE_AUTH_DOMAIN=ocean-air-care-ff757.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=ocean-air-care-ff757
VITE_FIREBASE_STORAGE_BUCKET=ocean-air-care-ff757.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=237486568893
VITE_FIREBASE_APP_ID=1:237486568893:web:9af5a645fa1293b2890ed4
VITE_FIREBASE_MEASUREMENT_ID=G-J62FC9EXCK
VITE_GOOGLE_PLACE_ID=ChIJ735DAJ1bUjoR-6B9hM5v6lo
VITE_WHATSAPP_NUMBER=919941103715
VITE_BUSINESS_NAME=Ocean Air Care
```

All VITE_ variables are inlined into the compiled bundle at build time.

8.3 Build & deploy

- Build command: yarn build (outputs frontend/dist/)
- Emergent handles CI/build/deploy on the Deploy action.
- Alternate configs shipped in repo: firebase.json + .firebaserc (Firebase Hosting), .github/workflows/deploy.yml (GitHub Pages).

8.4 Git history (module-by-module)

```
defe6d2  Firebase: add live project config
4db54e2  Firebase Hosting: configure site
2245c65  Code review fixes
ba26bd6  Module 8: PRD + test credentials
0667344  Module 7: App router + Firestore security rules + deployment workflow
ac002c9  Module 6: Admin flow
edc72e7  Module 5: Customer flow
d62e80d  Module 4: Shared UI components
b846558  Module 3: Auth wrapper
5a49b95  Module 2: Data layer + 540 seed reviews
b98ddf9  Module 1: Vite + Tailwind + PWA scaffold
```

Every commit represents one logical, self-contained module — a clean audit trail suitable for code review or future team onboarding.

9. Core user flows

9.1 Customer flow (public)

```
Scan QR -->  / (Landing)
                |
                v
            /services   -->  6 service tiles
                |
                v
            /rate/ac-service   -->  * * * * *
                |
                v
            /reviews/ac-service?rating=5
                |  (60 template cards, searchable, copy-to-clipboard)
                v
            "Open Google & paste"
                -->  https://search.google.com/local/writereview?placeid=ChIJ735DAJ1bUjoR-6B9hM5v6lo
                |  (new tab)
                v
            /thank-you
```

Every event fires an analytics increment (view, copy, google_click, whatsapp_click).

9.2 Admin flow (protected)

```
/admin/login  -->  Firebase Auth
                       |
                       v
                   /admin (Overview)
                       |
      +----------------+----------------+----------------+
      v                v                v                v
  /admin/reviews  /admin/categories  /admin/qr        Overview stats
    |
    +-- Add / Edit / Delete review
    +-- CSV Export --> downloads reviews-YYYY-MM-DD.csv
    +-- CSV Import --> replaces existing reviews
```

10. Security & privacy

• No customer PII collected — the app records anonymous analytics counters only. No emails, phone numbers, IPs, or names are stored.

• Firestore rules enforce public-read / admin-write on business data. Analytics writes are open (counter-only, no PII).

• HTTPS everywhere — Emergent hosting terminates TLS.

• Environment variables for the Firebase API key are safe to embed client-side — they are identifiers, not secrets. Access control is entirely through Firestore Security Rules.

• Admin credentials are managed by Firebase Auth (bcrypt-hashed in Google's infrastructure) — never touch this codebase.

11. Quality & testing

• Automated frontend smoke test executed via headless Chromium (Playwright).

• Iteration 1 result: 13/13 checks passing (100%).

• Covered: landing → services → rating → review library search + copy → Google CTA → thank-you → all admin CRUD flows → QR generator canvas render.

• Static analysis via the platform's deployment checker: PASS.

• Manual verification on production after Firebase configuration: admin sign-in, Firestore CRUD, real-time updates, Google review deep link — all verified end-to-end.

12. Known limitations & future work

Priority	Item	Notes
P1	Rating-aware template filtering	Show apologetic templates on 1–3 stars, enthusiastic on 4–5
P1	Multilingual (English + Tamil)	Chennai customer base — expected uplift in completion rate
P1	Freeform review composer	Textarea alongside templates for users who prefer their own words
P2	Cloud Function for analytics writes	Rate-limit & validate to harden the permissive analytics rule
P2	Twilio SMS reminders post-service	Typically doubles reviews-per-service ratio
P2	Multiple admin roles	Owner vs. manager
P2	Google Business Profile API	Fetch live star average / review count for dashboard

13. Appendix — key file references

Concern	File
Firebase init + config gate	frontend/src/lib/firebase.js
Auth (Firebase + local fallback)	frontend/src/lib/auth.jsx
Unified data layer	frontend/src/lib/data.js
540-template generator	frontend/src/data/seedReviews.js
Firestore security rules	firestore.rules
Firebase Hosting config	firebase.json, .firebaserc

Concern	File
GitHub Pages workflow	.github/workflows/deploy.yml
Product requirements	memory/PRD.md
Test credentials	memory/test_credentials.md

Prepared by the ReviewPro engineering agent · 8 January 2026